



Task: Prepare Data and Build a Logistic Regression Model

The “Three Strikes Before AI” rule:

Tagline: Try it yourself three times (documentation support is cool!). If you still can't solve it, then AI becomes your assistant, not your shortcut.

Useful links:

- https://pandas.pydata.org/docs/user_guide/10min.html
- https://pandas.pydata.org/docs/user_guide/index.html#user-guide
- <https://matplotlib.org/stable/tutorials/index.html>
- https://matplotlib.org/stable/users/explain/quick_start.html

1. Dataset cleaning

Let's simulate a daily debit or credit bank account transactions.

Step 1: Load the Dataset

- Load the file `daily_entries.csv` into a `DataFrame`.

Step 2: Inspect for Missing Values

- Check the dataset for any missing values (NaN) and understand where they occur.

2. Dataset preparation

Step 3: Handle Missing Data

Choose **one or more methods** to deal with missing values:

- **Drop rows with missing values.**
- **Replace missing values with the column average.**
- **Interpolate missing values** to estimate them based on surrounding data.

Step 4: Aggregate Customer Data

- Combine all entries from the column values for each customer into a single representation (e.g., one row per customer).
- Think about how to summarize these values (e.g., mean, sum, or keep as a sequence).



- Organize a **new dataframe** named '*accumulated balance*', so the cells are now related to the current balance of that customer in that given time.
 - Ex: value v of cell in time t is: $v_t = v_{t-1} + v_t$

Index	0	1	2	3	4	5
Previously	5	3	-12	23	-10	8
New value	5	8	-4	19	9	17

3. Data Visualization

Step 5: Visually checking the data

- Plot your new data using customers as different line colours

4. Data classification

Step 6: Prepare Features and Labels

- Define the input features (based on the aggregated values).
- Define the target label: whether the next transaction for the customer will be negative. (Spoiler alert: It should be the last entry).

Step 6: Split the Data

- Divide the dataset into **training** and **test** sets to evaluate your model.
- Implement your code as a class so you can specify the ratio for training and testing datasets (like 80% training, 20% test).

Step 7: Train a Logistic Regression Model

- Use the prepared features and labels to train two logistic regression models.
 - One to predict the value of the next transaction, based on previous one
 - One to predict whether the customer balance would be positive or negative in the next transaction
- The goal is to predict which customers will have a negative next transaction.